

# CONHECIMENTO E RACIOCÍNIO

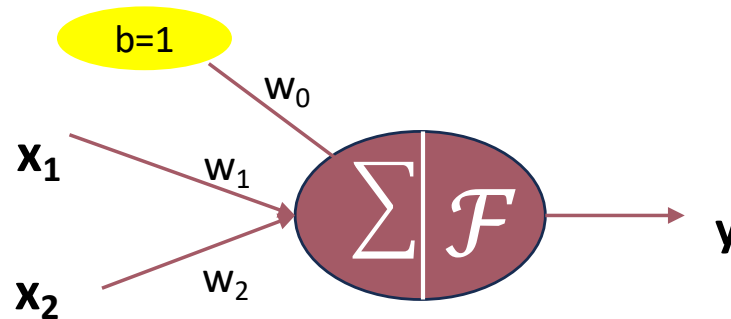
## 8 - Redes Neurais #2



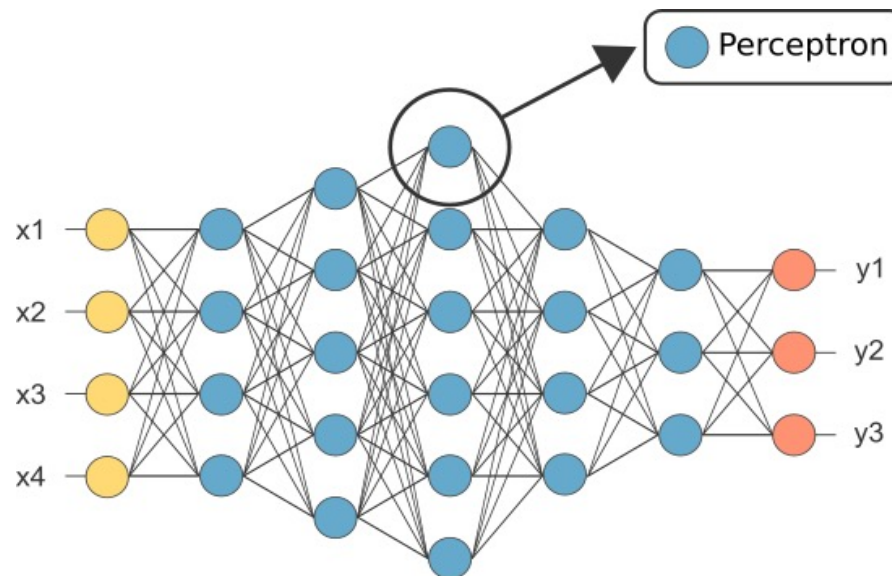
# Redes Neurais

## Tipos de redes neurais

- Perceptron



- Rede *Feedforward*: usa vários neurónios (perceptron) em diferentes camadas



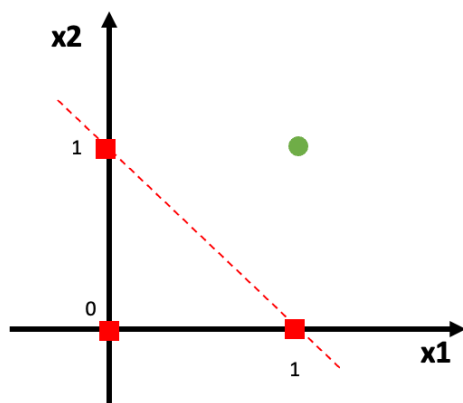
# Redes Neurais

## Redes neurais

- Problemas linearmente vs não linearmente separáveis

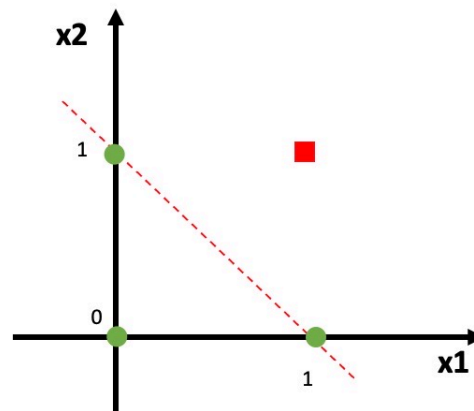
OR

| x1 | x2 | t |
|----|----|---|
| 0  | 0  | 0 |
| 0  | 1  | 1 |
| 1  | 0  | 1 |
| 1  | 1  | 1 |



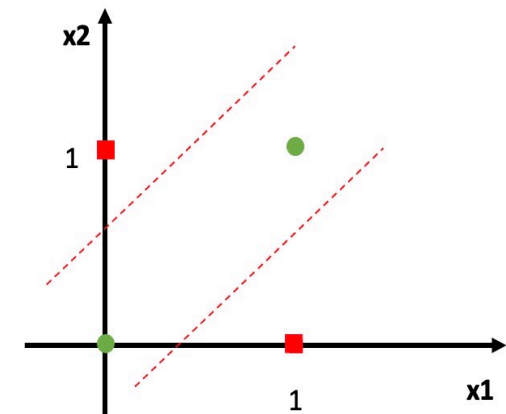
AND

| x1 | x2 | t |
|----|----|---|
| 0  | 0  | 0 |
| 0  | 1  | 0 |
| 1  | 0  | 0 |
| 1  | 1  | 1 |



XOR

| x1 | x2 | t |
|----|----|---|
| 0  | 0  | 0 |
| 0  | 1  | 1 |
| 1  | 0  | 1 |
| 1  | 1  | 0 |



$$x_2 = -x_1 \frac{w_1}{w_2} - b \frac{w_0}{w_2}$$

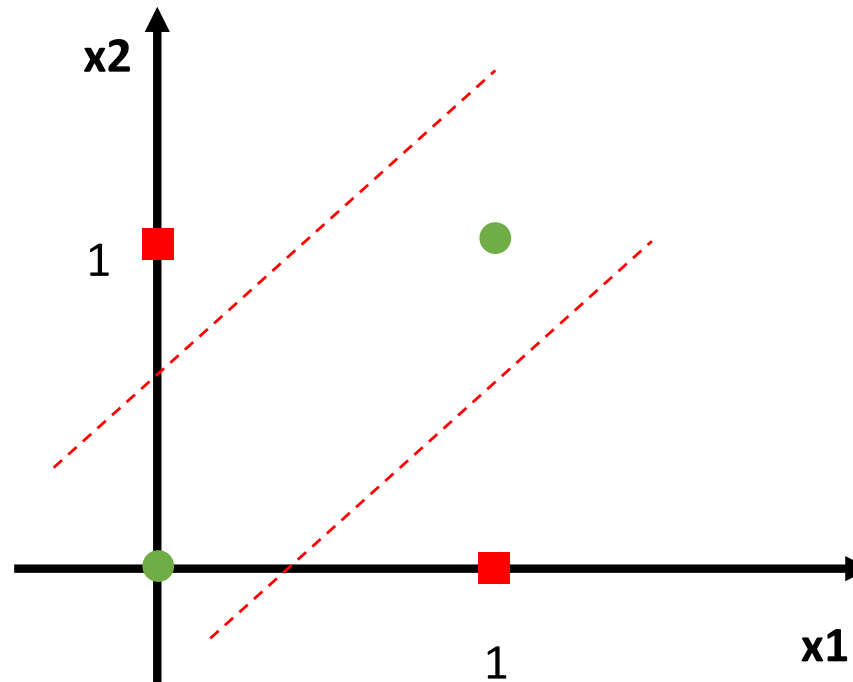
# Redes Neuronais

## Multilayer Perceptron

- Como resolver o problema XOR?
  - Usar mais do que um *perceptron*: camada escondida

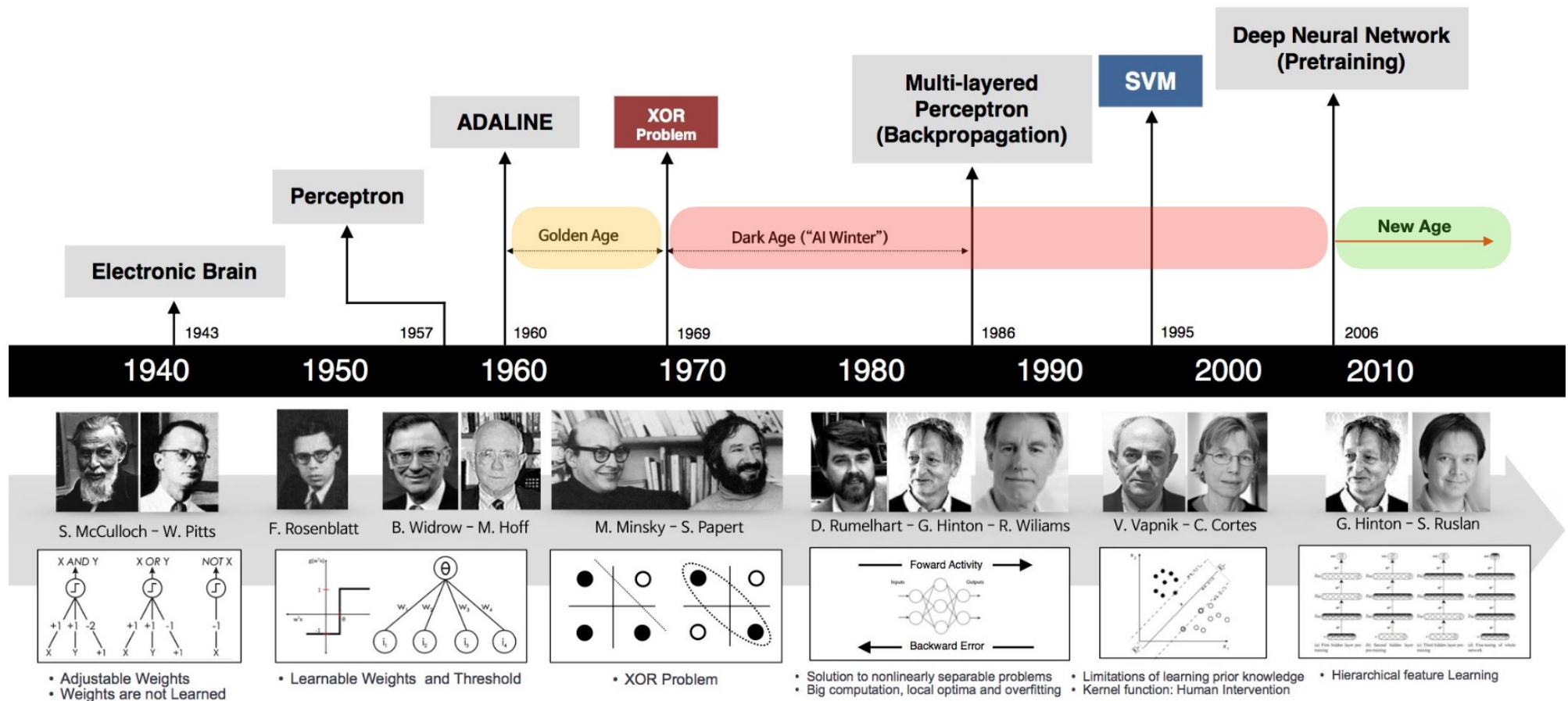
**XOR**

| $x_1$ | $x_2$ | $y$ |
|-------|-------|-----|
| 0     | 0     | 0   |
| 0     | 1     | 1   |
| 1     | 0     | 1   |
| 1     | 1     | 0   |



# Redes Neurais

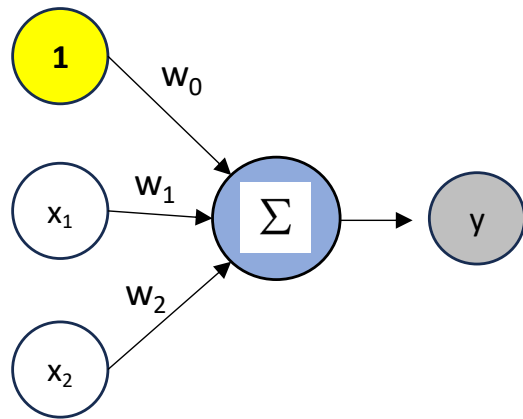
## Redes neuronais



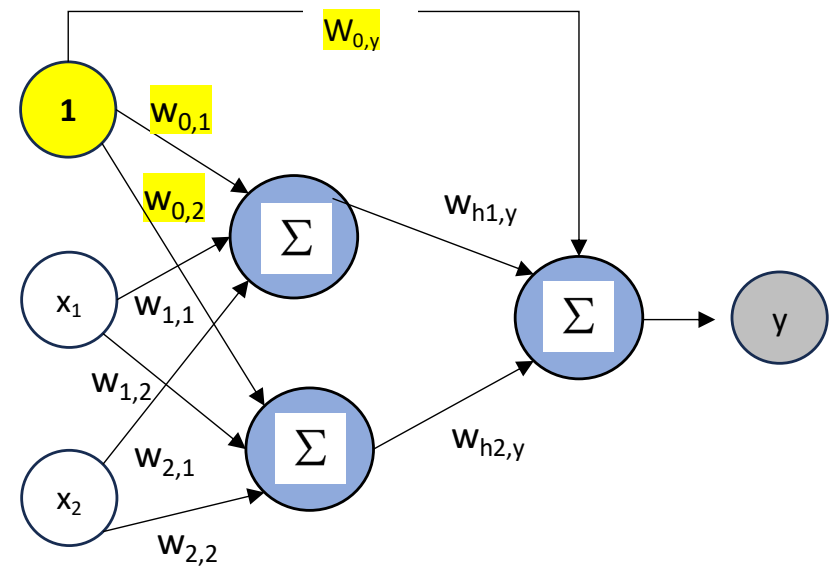
# Redes Neurais

## Multilayer Perceptron

- Como resolver o problema XOR?



Perceptron



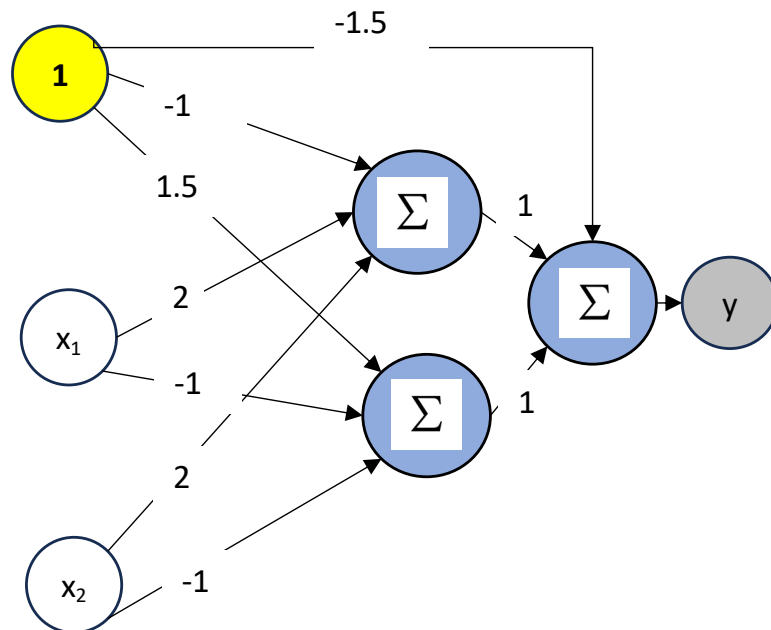
Multilayer perceptron: (h1 e h2)

# Redes Neurais

## Multilayer Perceptron

| x1 | x2 | t |
|----|----|---|
| 0  | 0  | 0 |
| 0  | 1  | 1 |
| 1  | 0  | 1 |
| 1  | 1  | 0 |

XOR



[0,0]

$$H1 = -1 + 0*2 + 0*2 = -1$$

$$H2 = 1.5 + 0*(-1) + 0*(-1) = 1.5$$

$$\text{Step}(H1) = 0$$

$$\text{Step}(H2) = 1$$

$$Y = -1.5 + (0*1) + (1*1) = -0.5$$

$$\text{Step}(Y) = 0$$

[1,0]

$$H1 = -1 + 1*2 + 0*2 = 1$$

$$H2 = 1.5 + 1*(-1) + 0*(-1) = 0.5$$

$$\text{Step}(H1) = 1$$

$$\text{Step}(H2) = 1$$

$$Y = -1.5 + (1*1) + (1*1) = 0.5$$

$$\text{Step}(Y) = 1$$

[0,1]

$$H1 = -1 + 0*2 + 1*2 = 1$$

$$H2 = 1.5 + 0*(-1) + 1*(-1) = 0.5$$

$$\text{Step}(H1) = 1$$

$$\text{Step}(H2) = 1$$

$$Y = -1.5 + (1*1) + (1*1) = 0.5$$

$$\text{Step}(Y) = 1$$

[1,1]

$$H1 = -1 + 1*2 + 1*2 = 3$$

$$H2 = 1.5 + 1*(-1) + 1*(-1) = -0.5$$

$$\text{Step}(H1) = 1$$

$$\text{Step}(H2) = 0$$

$$Y = -1.5 + (1*1) + (1*0) = -0.5$$

$$\text{Step}(Y) = 0$$

# Redes Neurais

## Redes neurais multilayer (feedforward)

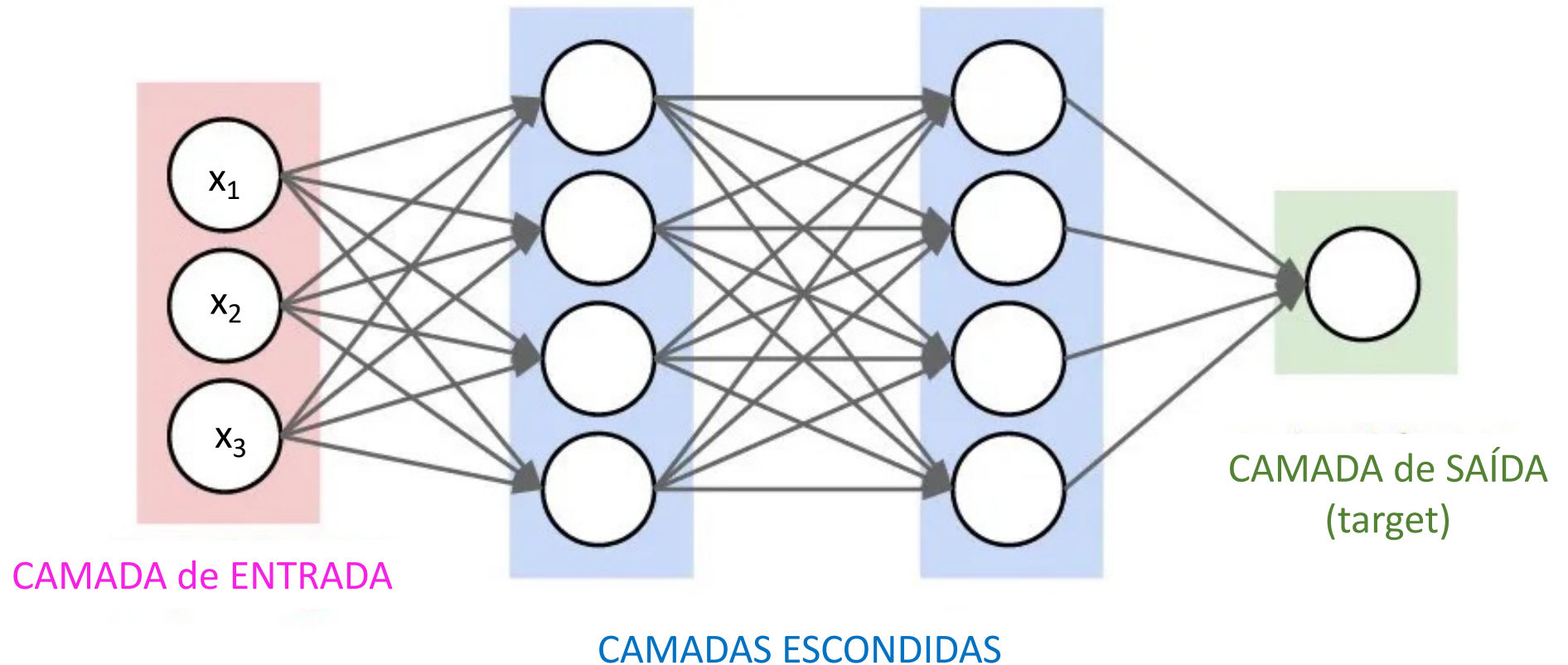
- Mais capacidade de aprendizagem:
  - *Perceptron com duas entradas*: divide as classes usando **uma reta**
    - Problemas linearmente separáveis
  - *Multilayer perceptron*: divide as classes usando **um plano ou hiperplano**
    - Problemas não linearmente separáveis
- *Redes Multilayer feedforward*:
  - **resolução de problemas mais complexos**
  - Tipo de ligações: *feedforward*
  - Diferentes funções de ativação (não lineares): *sigmoide*, por exemplo
  - Algoritmo de aprendizagem: *backpropagation*



# Redes Neuronais

## Redes neuronais multilayer (feedforward)

- Rede *Feedforward*



# Redes Neurais

## Redes neuronais multilayer (feedforward)

- Camada de entrada:
  - N° de neurónios (nós) = N° de variáveis de entrada (*features*)
- Camada de saída:
  - N° de neurónios (nós) = N° de variáveis de saída (*classes*)
- **TOPOLOGIA DA REDE:**
  - Número de camadas escondidas
  - Número de neurónios por camada

**Deve ser ajustado dependente do problema**

# Redes Neurais

## Redes neuronais multilayer (feedforward)

- Rede *Feedforward*: exemplo – iris dataset

4 entradas: medidas da pétala e sépala

3 saídas

|   | sepal length (cm) | sepal width (cm) | petal length (cm) | petal width (cm) | species |
|---|-------------------|------------------|-------------------|------------------|---------|
| 0 | 5.1               | 3.5              | 1.4               | 0.2              | setosa  |
| 1 | 4.9               | 3.0              | 1.4               | 0.2              | setosa  |
| 2 | 4.7               | 3.2              | 1.3               | 0.2              | setosa  |
| 3 | 4.6               | 3.1              | 1.5               | 0.2              | setosa  |
| 4 | 5.0               | 3.6              | 1.4               | 0.2              | setosa  |



Iris Versicolor



Iris Setosa



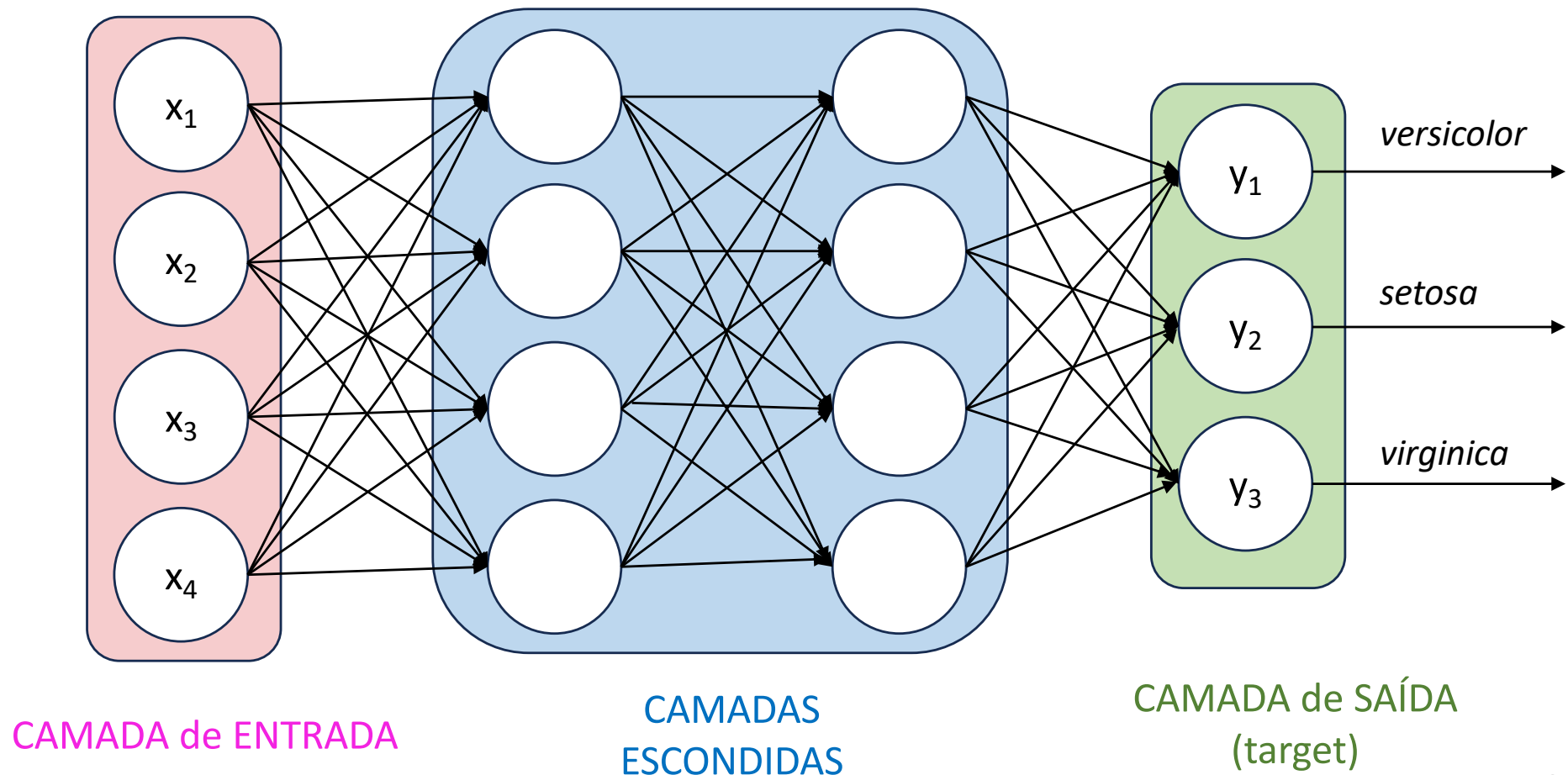
Iris Virginica

Exemplos de treino

# Redes Neurais

## Redes neuronais multilayer (feedforward)

- Exemplo de uma rede *Feedforward* para treinar o dataset da Iris:



# Redes Neurais

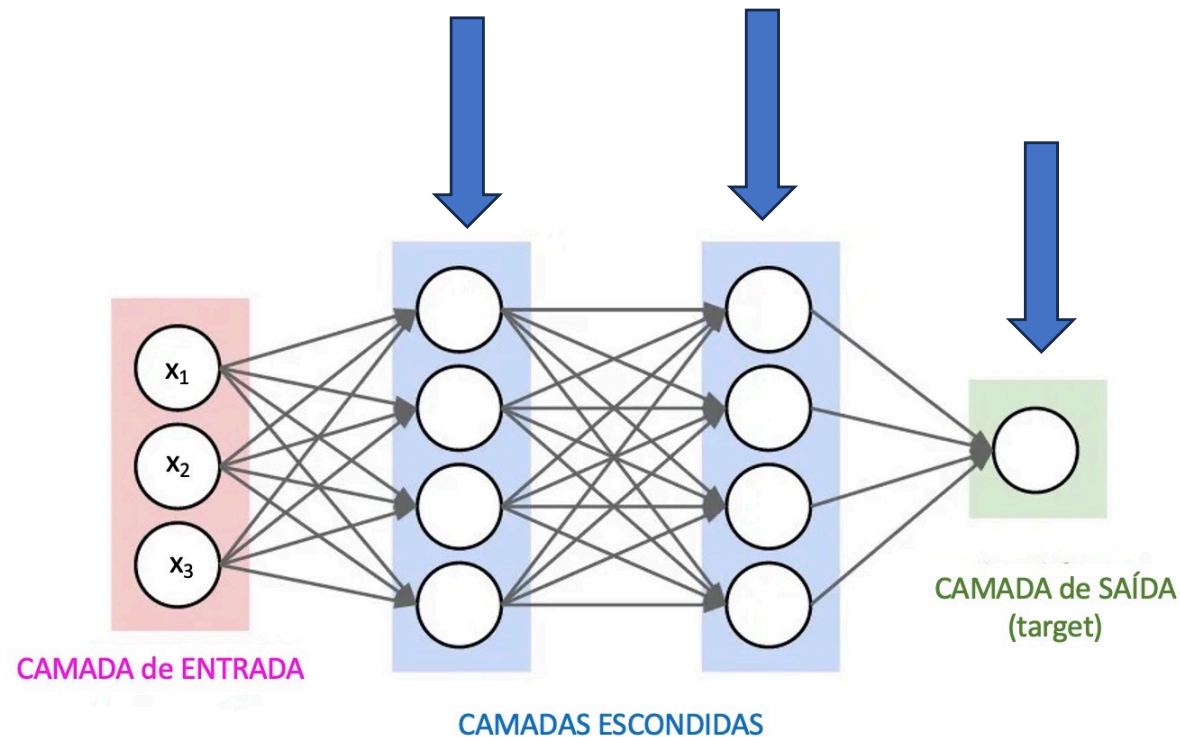
## Redes neuronais multilayer (feedforward)

- Cada neurónio:
  - $S = b * w_0 + \sum_{i=1}^n x_i w_i$
  - Output =  $F(S)$ 
    - $F$  é chamada de **função de ativação**
- Na camada de saída:
  - O valor de  $F(S)$  desta camada é comparado com o target (**erro**)
- Nas camadas escondidas:
  - O resultado é dado por uma função de ativação
  - O erro da camada de saída é usado para calcular os erros das camadas escondidas
- Usando um **algoritmo de treino** iterativo, **TODOS** os pesos da rede são ajustados até o erro ser zero ou ser atingido um critério de convergência

# Redes Neurais

## Redes neuronais multilayer (feedforward)

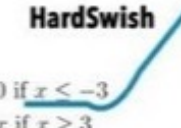
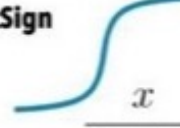
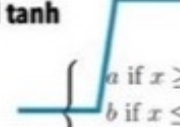
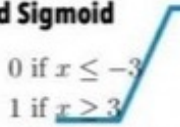
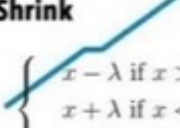
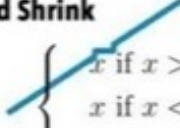
- **Funções de ativação:** sigmoide, RELU, tangente, softmax, ...
- Pode ser escolhida uma função de ativação diferente para cada camada escondida e para a camada de saída



# Redes Neurais

## Redes neurais multilayer (feedforward)

- Funções de ativação:

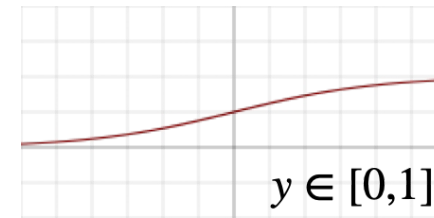
|                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ReLU</b><br><br>$\max(0, x)$                                                                                                      | <b>GELU</b><br><br>$\frac{x}{2} \left( 1 + \tanh \left( \sqrt{\frac{2}{\pi}} (x + ax^3) \right) \right)$                                                      | <b>PReLU</b><br><br>$\max(0, x)$                                                                                                             |
| <b>ELU</b><br><br>$\begin{cases} x & \text{if } x > 0 \\ \alpha(x \exp x - 1) & \text{if } x < 0 \end{cases}$                        | <b>Swish</b><br><br>$\frac{x}{1 + \exp -x}$                                                                                                                   | <b>SELU</b><br><br>$\alpha(\max(0, x) + \min(0, \beta(\exp x - 1)))$                                                                         |
| <b>SoftPlus</b><br><br>$\frac{1}{\beta} \log(1 + \exp(\beta x))$                                                                     | <b>Mish</b><br><br>$x \tanh \left( \frac{1}{\beta} \log(1 + \exp(\beta x)) \right)$                                                                           | <b>RReLU</b><br><br>$\begin{cases} x & \text{if } x \geq 0 \\ ax & \text{if } x < 0 \text{ with } a \sim \mathcal{R}(l, u) \end{cases}$      |
| <b>HardSwish</b><br><br>$\begin{cases} 0 & \text{if } x < -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$ | <b>Sigmoid</b><br><br>$\frac{1}{1 + \exp(-x)}$                                                                                                               | <b>SoftSign</b><br><br>$\frac{x}{1 +  x }$                                                                                                  |
| <b>Tanh</b><br><br>$\tanh(x)$                                                                                                      | <b>Hard tanh</b><br><br>$\begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$                              | <b>Hard Sigmoid</b><br><br>$\begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x \geq 3 \\ x/6 + 1/2 & \text{otherwise} \end{cases}$ |
| <b>Tanh Shrink</b><br><br>$x - \tanh(x)$                                                                                           | <b>Soft Shrink</b><br><br>$\begin{cases} x - \lambda & \text{if } x > \lambda \\ x + \lambda & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$ | <b>Hard Shrink</b><br><br>$\begin{cases} x & \text{if } x > \lambda \\ x & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$    |

# Redes Neurais

## Redes neuronais multilayer (feedforward)

Função de ativação: sigmoide

$$y = \frac{1}{1 + e^{-x}}$$



- É não linear
- É contínua e diferenciável (pode ser usada no *Gradiente Descendente*)
- É semelhante à função **Step**, o que sugere um comportamento parecido com o desta função

$$out = \begin{cases} 1, & \text{se } y \geq 0.5 \\ 0, & \text{se } y < 0.5 \end{cases}$$

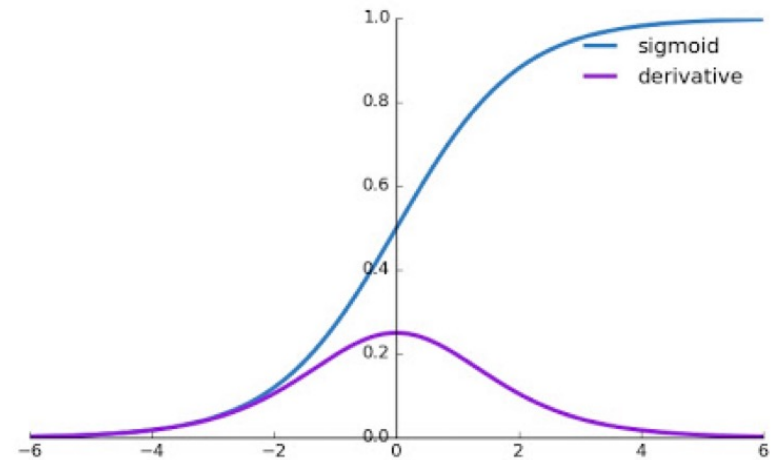


# Redes Neuronais

## Redes neuronais multilayer (feedforward)

### Função de ativação: sigmoide

$$y = \frac{1}{1 + e^{-x}}$$



- Derivada da função sigmoide:

$$\mathbf{sig}'(x) = \mathbf{sig}(x) \times (1 - \mathbf{sig}(x))$$

# Redes Neurais

## Redes neurais multilayer (feedforward)

- Algoritmos de treino
  - Regra delta
  - Gradiente Descendente
  - Backpropagation
- Em qualquer deles:
  - São aplicadas as entradas que constituem exemplos de treino
  - A cada exemplo de treino corresponde um valor alvo (target) que se compara com um resultado da rede: calcular o erro
  - Reajustam-se os pesos usando o erro
  - Terminado o treino, em presença de exemplos diferentes dos utilizados, a rede neuronal deverá responder corretamente (generalização)

# Redes Neurais

## Redes neuronais multilayer (feedforward)

- Regra Delta (ver aula anterior)
  - Usada no *perceptron*
  - Os pesos  $w_i$  são inicializados com valores aleatórios
  - Para cada exemplo, a alteração dos pesos  $w_i$  é feita segundo a regra:

$$w_i = w_i + \Delta w_i$$

$$\Delta w_i = \eta (t - y) x_i$$

$t$  = *target* para o exemplo actual

$y$  = *output* (valor da saída do perceptron)

$x_i$  = *input* ao qual foi aplicado o atributo  $i$  do exemplo

$\eta$  = *coeficiente de aprendizagem*

# Redes Neuronais

## Redes neuronais multilayer (feedforward)

- Gradiente Descendente

- Para iniciar a dedução do algoritmo, há que definir uma medida do erro observado entre um valor target e a saída do neurónio
- Seja a Soma do Erro Quadrático (SSE) para o exemplo de treino **d**:

$$E_d = \frac{1}{2} \sum_{d \in D} (t_d - y_d)^2$$

D = conjunto de exemplos de treino

$t_d$  = target para o exemplo  $d$

$o_d$  = saída do neurónio para o exemplo  $d$

NOTA: Enquanto  $E \geq 0$ , minimizar  $(1/2)E$  é equivalente a minimizar  $E$ . Para facilitar a derivação foi introduzido o componente  $1/2$

# Redes Neurais

## Redes neurais multilayer (feedforward)

- O gradiente de  $E(w_1, w_2, \dots, w_n)$  é dado pelas *derivadas parciais*:

$$\nabla E(w_0, w_1, \dots, w_n) = \left[ \frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n} \right]$$

- Como pretendemos uma diminuição, a expressão do gradiente terá de ser multiplicada por  $-1$
- No *Gradiente Descendente* os pesos  $w$  são atualizados assim:

$$w_i = w_i + \Delta w_i \text{ com } \Delta w_i = -\eta \frac{\partial E}{\partial w_i} \quad (\text{eq. 1})$$

- Agora, há que determinar a expressão de  $\frac{\partial E}{\partial w_i}$

# Redes Neurais

## Redes neuronais multilayer (feedforward)

- Como a função do erro é  $E_d = \frac{1}{2} \sum_{d \in D} (t_d - y_d)^2$
- Derivando, temos:

$$\frac{\partial E}{\partial w_i} = \frac{\partial \frac{1}{2} \sum_{d \in D} (t_d - y_d)^2}{\partial w_i} = \frac{1}{2} \sum_{d \in D} \frac{\partial (t_d - y_d)^2}{\partial w_i} = \frac{1}{2} \sum_{d \in D} 2(t_d - y_d)^1 \frac{\partial (t_d - y_d)}{\partial w_i}$$

$$= \sum_{d \in D} (t_d - y_d) \frac{\partial (t_d - y_d)}{\partial w_i} =$$

$$= \sum_{d \in D} (t_d - y_d)(-x_{i,d}) \text{ (eq. 2)}$$

# Redes Neuronais

## Redes neuronais multilayer (feedforward)

Substituindo eq.2 na eq.1 obtém-se a **correção  $\Delta w_i$  dos pesos  $w_i$**  em cada passo do algoritmo:

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i} \quad (\text{eq. 1})$$

$$\frac{\partial E}{\partial w_i} = \sum_{d \in D} (t_d - y_d)(-x_{i,d}) \quad (\text{eq. 2})$$

$$\Delta w_i = \eta \sum_{d \in D} (t_d - y_d) x_{i,d} \quad (\text{eq. 3})$$

$d$  é o índice do exemplo

$D$  é o conjunto de todos os exemplos

# Redes Neurais

## Redes neuronais multilayer (feedforward)

Resumo do Gradiente Descendente:

1. Inicializar os pesos  $w$  aleatoriamente
2. Aplicar todos os exemplos de treino ao neurónio linear
3. Calcular o  $\Delta w_i$  para cada peso  $w$  segundo a equação 3
4. Ajustar o valor de cada peso  $w_i$  usando a equação  $w_i = w_i + \Delta w_i$

Voltar ao passo 2 até que o erro seja suficientemente baixo

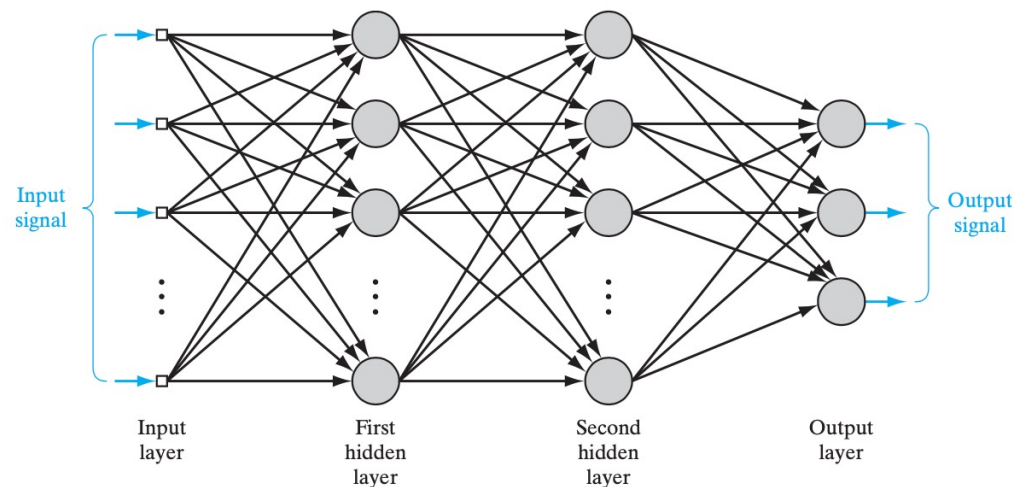


# Redes Neurais

## Redes neuronais multilayer (feedforward)

Os algoritmos de treino explicados mostram o funcionamento para ajustar os pesos associados a **UM** neurónio

Numa rede *feedforward* temos várias camadas, vários neurónios todos interligados: muitos pesos!!



- O algoritmo de treino para ajustar **TODOS** os pesos de uma rede *feedforward* é o ***backpropagation***

# Redes Neurais:

## Redes neurais multilayer (feedforward)

**Aprendizagem** de uma rede neuronal feedforward: ajustar todos os pesos da rede

- Algoritmo de treino: **backpropagation**
  - Proposto nos anos 60 e popularizado 1989 por Rumelhart, Hinton and Williams no artigo "Learning representations by back-propagating errors".
- Usado para treinar uma rede neuronal:
  - Iterativamente calcular a(s) saída(s) da rede (**forward pass**), calcular o erro e ajustar os pesos retropropagando o erro para trás (**backward pass ou backpropagation**)
  - Objetivo: minimizar a função de custo ajustando os pesos da rede. O nível de ajustamento é determinado pelos **gradientes da função de custo** em relação a esses pesos

# Redes Neurais:

## Redes neurais multilayer (feedforward)

### Backpropagation

- Determina qual a influência que cada peso tem na saída desejada
- Para cada exemplo de treino ajusta os pesos de forma a
  - Reforçar as ligações que vão ativar a saída desejada
  - Inibir as ligações das saídas erradas
    - REGRA DE HEBB
- Repetir o processo de ajuste dos pesos para TODOS os exemplos de treino
- Objetivo: minimizar o erro total para todos os exemplos
  - GRADIENTE DESCENDENTE

# Redes Neurais:

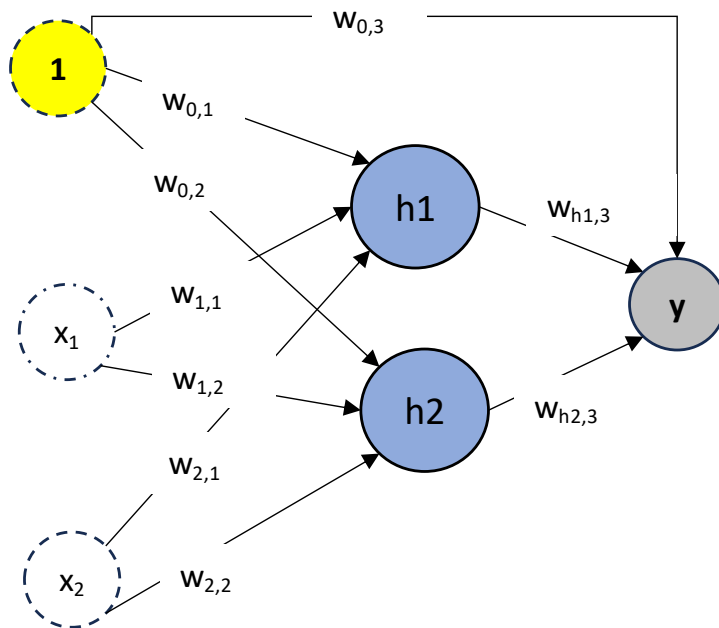
## Redes neuronais multilayer (feedforward)

- Algoritmo de treino: **backpropagation**
  - Entradas
    - Exemplos de treino  $(x, t)$ 
      - $x$  – variáveis de entrada
      - $t$  – saída desejada
    - Coeficiente de aprendizagem -  $\eta$  (0.01, por exemplo)
    - Número de nós na camada de entrada –  $n_i$
    - Número de nós na(s) camada(s) escondida(s) -  $n_{\text{hidden}}$
    - Número nós na camada de saída –  $n_{\text{out}}$
    - $x_i$ : entrada do neurónio  $i$
    - $w_{ij}$ : peso da ligação entre o neurónio  $i$  e o neurónio  $j$
  - Saída: valores dos pesos ajustados

# Redes Neurais:

## Redes neuronais multilayer (feedforward)

- **Backpropagation:** ajustar os pesos de uma rede feedforward



$$X = \begin{bmatrix} x_{1,1} & x_{1,2} & x_{1,3} & x_{1,4} \\ x_{2,1} & x_{2,2} & x_{2,3} & x_{2,4} \end{bmatrix}$$

Nº de linhas: número de entradas;  
Nº de colunas: exemplos de treino

$$t = [t_1 \quad t_2 \quad t_3 \quad t_4]$$

Saída desejada para cada ex. de treino

$$y = [y_1 \quad y_2 \quad y_3 \quad y_4]$$

Saída obtida para cada ex. de treino

$$W_0 = \begin{bmatrix} w_{0,1} \\ w_{0,2} \\ w_{0,3} \end{bmatrix}$$

Vetor de pesos associados ao *bias*

$$W = \begin{bmatrix} w_{1,1} & w_{2,1} \\ w_{1,2} & w_{2,2} \\ w_{h1,3} & w_{h2,3} \end{bmatrix}$$

Matriz de pesos de todas as ligações

# Redes Neurais:

## backpropagation

### Derivação no algoritmo de backpropagation:

- Usa o algoritmo do gradiente descendente
- O gradiente descendente é iterado para cada exemplo de treino  $d$ , minimizando o erro  $E_d$  para esse exemplo
- Para cada exemplo de treino  $d$ , todos os pesos  $w_{ji}$  são atualizados adicionando  $\Delta w_{ji}$ :  $w_{ji} = w_{ji} + \Delta w_{ji}$

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

Erro: função de custo:

$$E_d = \frac{1}{2} \sum_{k=1}^n (t_k - y_k)^2$$

k: todas as saídas da rede

Minimização do erro = Gradiente descendente

# Redes Neurais:

## backpropagation

**Algoritmo backpropagation** (usando a função de ativação sigmoide)

Obter conjunto de dados supervisionado  $D$  com  $d_i = (x_i, t_i)$

Inicializar os pesos de forma aleatória (valores baixos)

**Repetir**

2.1. Para cada  $d_i$  de  $D$

2.1.1. Propagar a entrada pela rede e calcular a saída ( $y_k$ ) para o nó  $k$

← **Forward pass**

$$o_{hj} = \text{sig}(\sum x_{ji} w_{ih_j})$$

$$y_k = \text{sig}(\sum o_{hi} w_{hik})$$

2.1.2. Retro propagar o erro para as camadas anteriores

← **Backward pass**

Calcular o erro para o nó de saída  $k$ :

$$\delta_k = y_k (1 - y_k) (t_k - y_k)$$

Calcular o erro para o nó escondido  $h_i$ :

$$\delta_{hi} = o_{hi} (1 - o_{hi}) \cdot \sum w_{hk} \delta_k$$

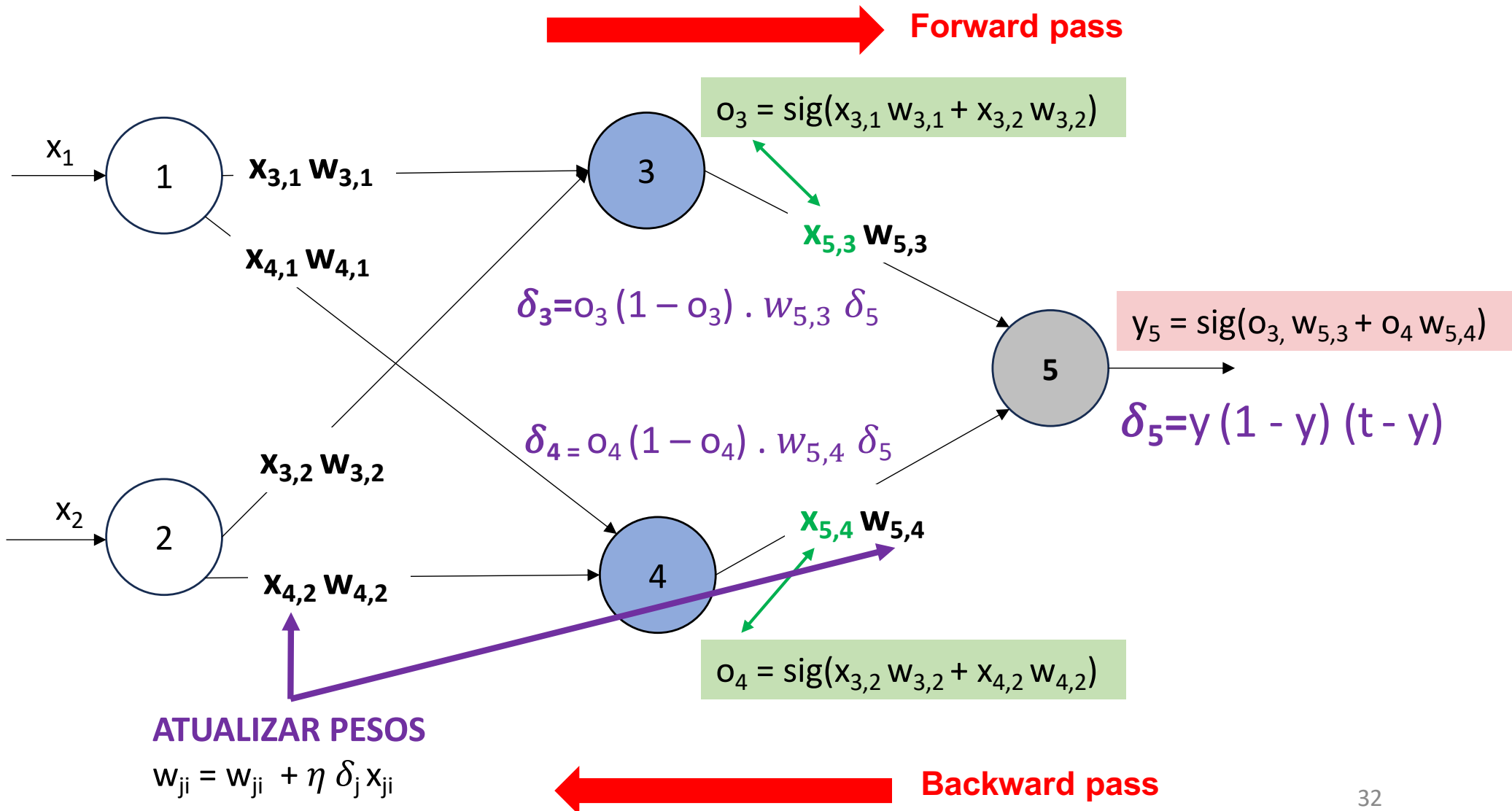
Atualizar os pesos de acordo com:

$$w_{ji} = w_{ji} + \Delta w_{ji} \quad \text{Com } \Delta w_{ji} = \eta \delta_j x_{ji}$$

Até condição de paragem

Devolver pesos  $W$

# Redes Neurais: backpropagation

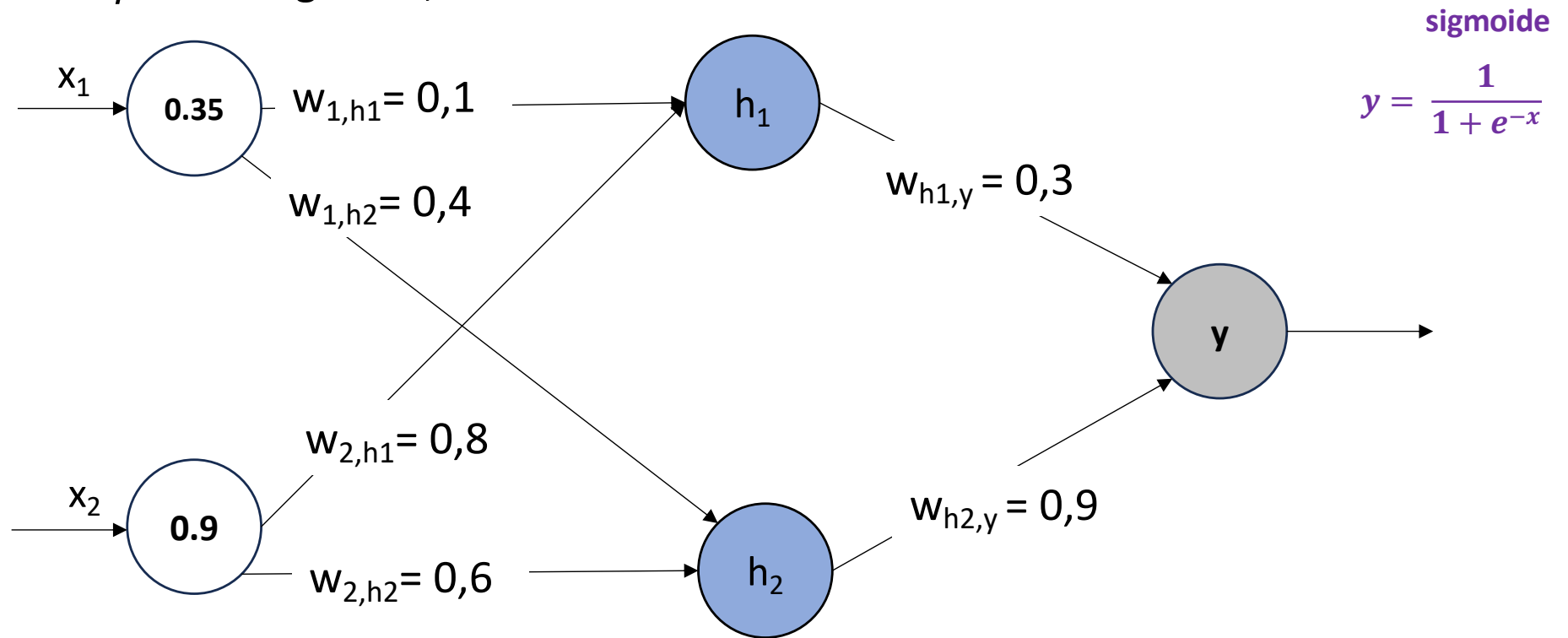




# Redes Neurais

## Redes neuronais multilayer (feedforward)

- Exemplo: fazer um **forward pass**, retropropagar o erro e repetir um **forward pass**
- Usar  $\eta = 1$  e target = 0,5



# Redes Neurais

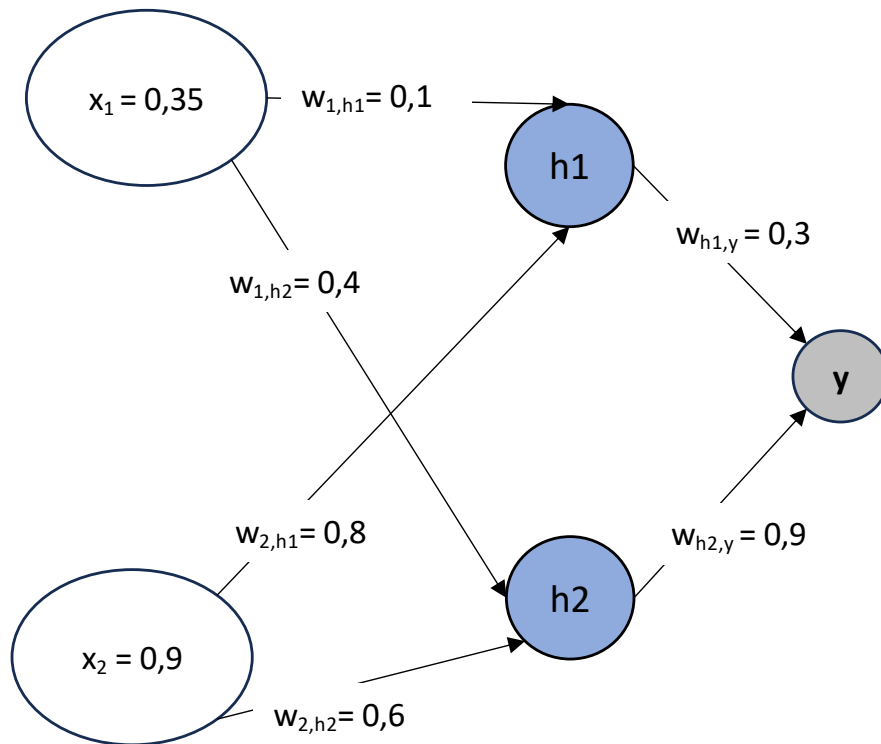
## Redes neuronais multilayer (feedforward)

$$y = \frac{1}{1 + e^{-x}} \quad \text{sigmoide}$$

$$\delta_y = y(1 - y)(t - y)$$

$$\delta_h = o_h(1 - o_h) \cdot \sum w_{hy} \delta_y$$

- Exemplo:  $\eta = 1$  e target = 0,5



$$\text{ERRO} = (\text{target} - y) = 0,5 - 0,69 = -0,19$$

1º Forward pass:

$$H1 = 0,35 * 0,1 + 0,9 * 0,8 = 0,755 \rightarrow \text{sig}(0,755) = 0,68$$

$$H2 = 0,35 * 0,4 + 0,9 * 0,6 = 0,68 \rightarrow \text{sig}(0,68) = 0,66$$

$$Y = 0,68 * 0,3 + 0,66 * 0,9 = 0,798 \rightarrow \text{sig}(0,798) = 0,69$$

$$\delta = 0,69 * (1 - 0,69) * (0,5 - 0,69) = -0,0406$$

2º Backward pass:

$$\delta_{h1} = 0,68 * (1 - 0,68) * (0,3 * (-0,0406)) = -0,00265$$

$$\delta_{h2} = 0,66 * (1 - 0,66) * (0,9 * (-0,0406)) = -0,0082$$

3º atualizar pesos:  $w_{ji} = w_{ji} + \eta \delta_j x_{ji}$ , com  $\eta = 1$

$$w_{h1,y} = 0,3 - 1 * 0,0406 * 0,68 = 0,272$$

$$w_{h2,y} = 0,9 - 1 * 0,0406 * 0,66 = 0,873$$

$$w_{1,h1} = 0,1 - 1 * 0,00265 * 0,35 = 0,099$$

$$w_{1,h2} = 0,4 - 1 * 0,0082 * 0,35 = 0,397$$

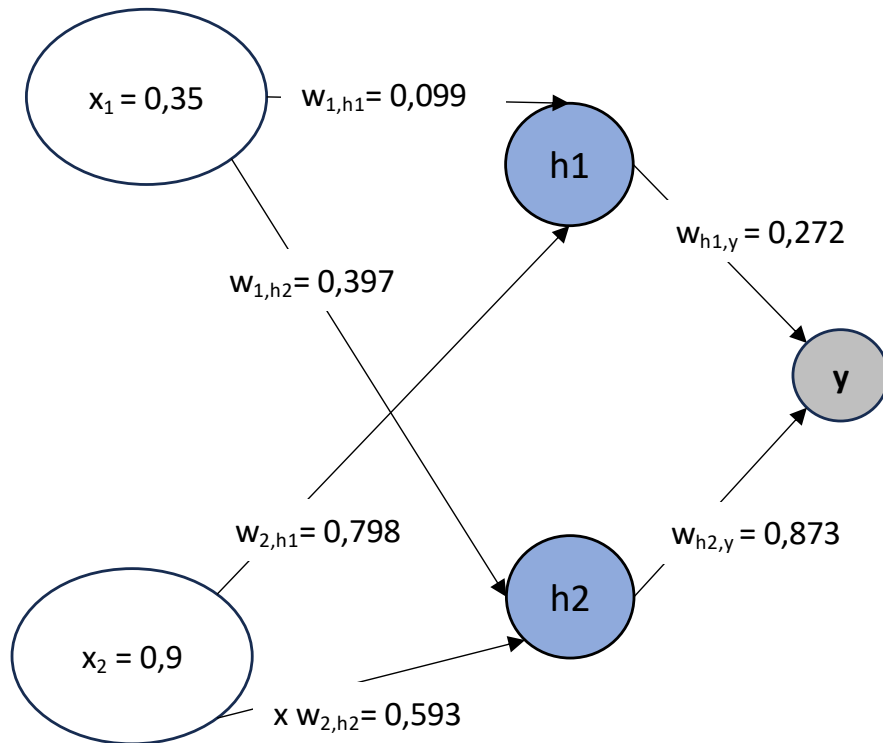
$$w_{2,h1} = 0,8 - 1 * 0,00265 * 0,9 = 0,798$$

$$w_{2,h2} = 0,6 - 1 * 0,0082 * 0,9 = 0,593$$

# Redes Neurais

## Redes neuronais multilayer (feedforward)

- Exemplo:  $\eta = 1$  e target = 0,5



2º Forward pass: o erro diminui??

$$H1 = 0,35 * 0,099 + 0,9 * 0,798 = 0,7525 \rightarrow \text{sig}(0,7525) = 0,6797$$

$$H2 = 0,35 * 0,397 + 0,9 * 0,593 = 0,672 \rightarrow \text{sig}(0,672) = 0,662$$

$$Y = 0,6897 * 0,272 + 0,662 * 0,873 = 0,7632 \rightarrow \text{sig}(0,7631) = 0,682$$

$$\text{ERRO} = (\text{target} - y) = 0,5 - 0,682 = -0,182$$

↑  
Erro diminui lentamente  
(gradiente descendente)

# Redes Neurais

## Redes neuronais multilayer (feedforward)

- O *backpropagation* confere às redes *feedforward* grande capacidade de aprendizagem
- Estas redes neuronais são muito usadas em problemas de classificação, reconhecimento de sons, imagens, padrões, caracteres, ...
- Como otimizar o desempenho de uma rede neuronal?
  - Escolher outras funções de ativação (além da sigmoide)
  - Escolher o número de camadas escondidas
  - Escolher o número de neurónios por camada escondida
  - Escolher coeficiente de aprendizagem
  - A segmentação do *dataset* usada no treino da rede também pode influenciar o desempenho da rede

# Redes Neurais

## Redes neurais multilayer (feedforward)

- **Outras redes neurais**

- Redes recorrentes (RNN)
- Long short-term memory (LSTM)
- Gated recurrent network (GRU)
- Convolutional neural network (CNN) ou Deep convolutional network (DCN)
- Generative adversarial network (GAN)
- ....
- E muitas muitas outras!!!!

# DEEP LEARNING

# Redes Neurais

## Redes neuronais multilayer (feedforward)

- **Aprendizagem supervisionada usando redes neuronais**
  - Obter conjunto de dados supervisionados (dataset)
  - Construir a rede neuronal
  - Treinar a rede neuronal:
    - Usar e avaliar o desempenho no treino para diferentes configurações
      - Número de camadas escondidas
      - Número de neurónios por camada
      - Diferentes funções de ativação
      - Diferentes algoritmos de treino
  - Testar a rede neuronal
  - Avaliar o desempenho da rede no teste

# Redes Neurais

## Redes neurais multilayer (feedforward)

- **Treinar e testar a rede neuronal:**
  - Usar o dataset completo para treinar a rede
  - Gerar/obter novos exemplos para testar a rede
    - Em muitos problemas obter novos exemplos não é fácil ou viável. Solução?
      - Usar o dataset para ambas as tarefas
        - Split do dataset em três partes: train/validation/test

# Redes Neurais

## Redes neurais multilayer (feedforward)

- **Métricas de avaliação do desempenho de uma rede neuronal**
  - **Erro da rede**
    - Minimizar o erro total da rede para todas as entradas
  - ***Accuracy* (taxa de acerto)**
    - Maximizar a taxa de acerto da rede para todas as entradas

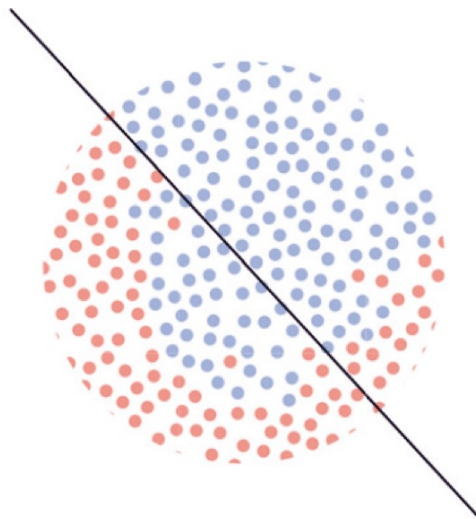


# Redes Neuronais

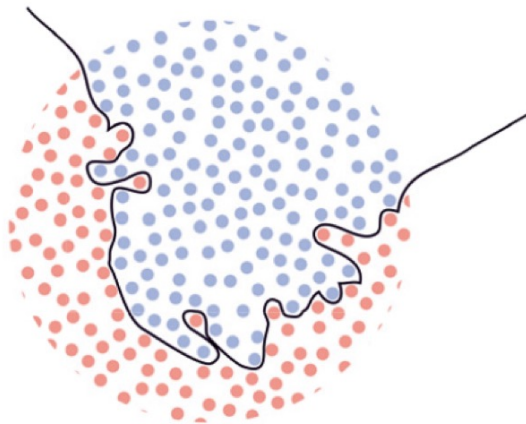
## Redes neuronais multilayer (feedforward)

- **Generalização vs *Overfitting***

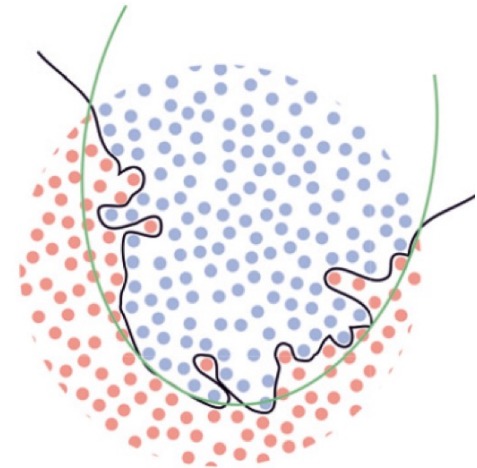
- Uma rede com um desempenho muito bom no treino, muitas vezes tem um fraco desempenho no teste: overfitting



Under



Over



Good

# Redes Neurais

## Redes neuronais multilayer (feedforward)

- Exemplo de uma rede *feedforward* para aprender a reconhecer os 9 dígitos
- Conjunto de dados supervisionados **D** (16 x 10 exemplos)

Dados de entrada (todos os exemplos)

Target (saída desejada)

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | → | 0 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | → | 1 |
| 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | → | 2 |
| 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | → | 3 |
| 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | → | 4 |
| 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | → | 5 |
| 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | → | 6 |
| 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | → | 7 |
| 8 | 8 | 8 | 8 | 8 | 8 | 8 | 8 | 8 | 8 | 8 | 8 | 8 | 8 | 8 | 8 | → | 8 |
| 9 | 9 | 9 | 9 | 9 | 9 | 9 | 9 | 9 | 9 | 9 | 9 | 9 | 9 | 9 | 9 | → | 9 |

# Redes Neurais

## Redes neurais multilayer (feedforward)

- **Construir a rede neuronal:**
  - Construir a rede: camada de entrada, camadas escondidas, camada de saída
- **Treinar a rede neuronal:**
  - Usar uma parte dos exemplos para treino (70%)
  - Inicializar a rede com pesos aleatórios
  - Usar o algoritmo de treino (*backpropagation*) para ajustar os pesos até o erro ser zero (ou outro critério)
  - Avaliar o desempenho da rede na fase de treino
- **Testar a rede neuronal**
  - Usar os 30% dos exemplos não usados no treino
  - Ou desenhar novos dígitos (diferentes dos que foram usados no treino)
  - Usar a rede treinada e ver a classificação dada
  - Avaliar o desempenho da rede na fase de teste

# Redes Neurais

## Redes neuronais multilayer (feedforward)

Avaliar o desempenho da rede na fase de treino e de teste

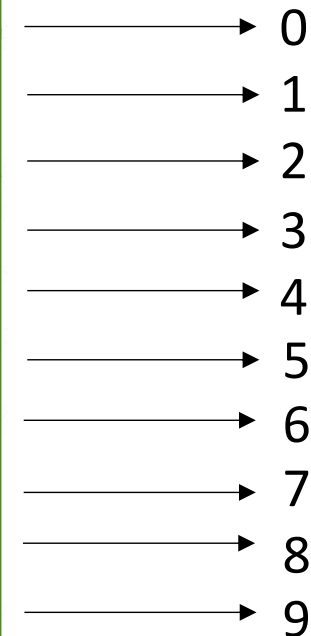
11 exemplos de treino



6 exemplos de teste



Target (saída desejada)



# Redes Neurais

## Redes neuronais multilayer (feedforward)

- A rede é treinada para cada dígito (11 exemplos de cada dígito)
  - Entrada da rede: dígito
    - Um dígito é uma matriz de valores
  - Target: dígito que pretendemos classificar



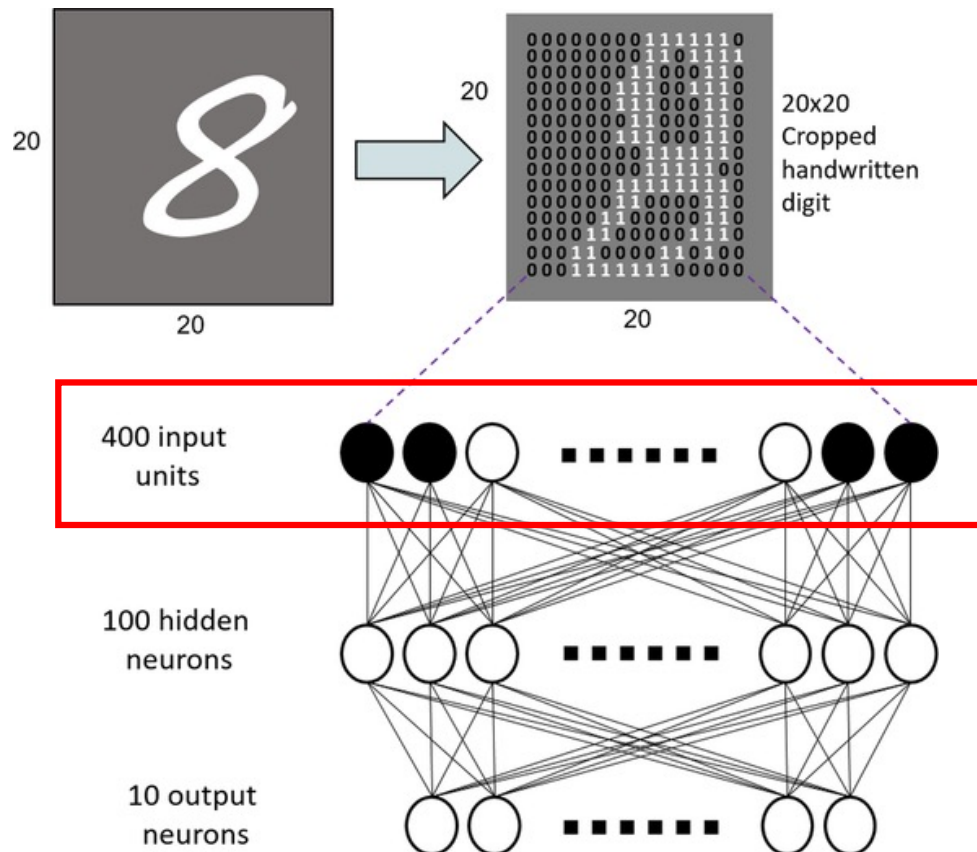
|    |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 0  | 2   | 15  | 0   | 0   | 11  | 10  | 0   | 0   | 0   | 9   | 9   | 0   | 0   | 0   |
| 0  | 0   | 0   | 4   | 60  | 157 | 236 | 255 | 255 | 177 | 95  | 61  | 32  | 0   | 0   |
| 0  | 10  | 16  | 119 | 238 | 255 | 244 | 245 | 243 | 250 | 249 | 255 | 222 | 103 | 10  |
| 0  | 14  | 170 | 255 | 255 | 244 | 254 | 255 | 253 | 245 | 255 | 249 | 253 | 251 | 124 |
| 2  | 98  | 255 | 228 | 255 | 251 | 254 | 211 | 141 | 116 | 122 | 215 | 251 | 238 | 255 |
| 13 | 217 | 243 | 255 | 155 | 33  | 226 | 52  | 2   | 0   | 10  | 13  | 232 | 255 | 255 |
| 16 | 229 | 252 | 254 | 49  | 12  | 0   | 0   | 7   | 7   | 0   | 70  | 237 | 252 | 235 |
| 6  | 141 | 245 | 255 | 212 | 25  | 11  | 9   | 3   | 0   | 115 | 236 | 243 | 255 | 137 |
| 0  | 87  | 252 | 250 | 248 | 215 | 60  | 0   | 1   | 121 | 252 | 255 | 248 | 144 | 6   |
| 0  | 13  | 115 | 255 | 255 | 245 | 255 | 182 | 181 | 248 | 252 | 242 | 208 | 36  | 0   |
| 1  | 0   | 5   | 117 | 251 | 255 | 241 | 255 | 247 | 255 | 241 | 162 | 17  | 0   | 7   |
| 0  | 0   | 0   | 4   | 58  | 251 | 255 | 246 | 254 | 253 | 255 | 120 | 11  | 0   | 1   |
| 0  | 0   | 4   | 97  | 255 | 255 | 255 | 248 | 252 | 255 | 244 | 255 | 182 | 10  | 0   |
| 0  | 22  | 206 | 252 | 246 | 251 | 241 | 100 | 24  | 113 | 255 | 245 | 255 | 194 | 9   |
| 0  | 111 | 255 | 242 | 255 | 158 | 24  | 0   | 0   | 6   | 39  | 255 | 232 | 230 | 56  |
| 0  | 218 | 251 | 250 | 137 | 7   | 11  | 0   | 0   | 0   | 2   | 62  | 255 | 250 | 125 |
| 0  | 173 | 255 | 255 | 101 | 9   | 20  | 0   | 13  | 3   | 13  | 182 | 251 | 245 | 61  |
| 0  | 107 | 251 | 241 | 255 | 230 | 98  | 55  | 19  | 118 | 217 | 248 | 253 | 255 | 52  |
| 0  | 18  | 146 | 250 | 255 | 247 | 255 | 255 | 255 | 249 | 255 | 240 | 255 | 129 | 0   |
| 0  | 0   | 23  | 113 | 215 | 255 | 250 | 248 | 255 | 255 | 248 | 248 | 118 | 14  | 12  |
| 0  | 0   | 6   | 1   | 0   | 52  | 153 | 233 | 255 | 252 | 147 | 37  | 0   | 0   | 4   |
| 0  | 0   | 5   | 5   | 0   | 0   | 0   | 0   | 14  | 1   | 0   | 6   | 6   | 0   | 0   |

|    |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 0  | 2   | 15  | 0   | 0   | 11  | 10  | 0   | 0   | 0   | 9   | 9   | 0   | 0   | 0   |
| 0  | 0   | 0   | 4   | 60  | 157 | 236 | 255 | 255 | 177 | 95  | 61  | 32  | 0   | 0   |
| 0  | 10  | 16  | 119 | 238 | 255 | 244 | 245 | 243 | 250 | 249 | 255 | 222 | 103 | 10  |
| 0  | 14  | 170 | 255 | 255 | 244 | 254 | 255 | 253 | 245 | 255 | 249 | 253 | 251 | 124 |
| 2  | 98  | 255 | 228 | 255 | 251 | 254 | 211 | 141 | 116 | 122 | 215 | 251 | 238 | 255 |
| 13 | 217 | 243 | 255 | 155 | 33  | 226 | 52  | 2   | 0   | 10  | 13  | 232 | 255 | 255 |
| 16 | 229 | 252 | 254 | 49  | 12  | 0   | 0   | 7   | 7   | 0   | 70  | 237 | 252 | 235 |
| 6  | 141 | 245 | 255 | 212 | 25  | 11  | 9   | 3   | 0   | 115 | 236 | 243 | 255 | 137 |
| 0  | 87  | 252 | 250 | 248 | 215 | 60  | 0   | 1   | 121 | 252 | 255 | 248 | 144 | 6   |
| 0  | 13  | 115 | 255 | 255 | 245 | 255 | 182 | 181 | 248 | 252 | 242 | 208 | 36  | 0   |
| 1  | 0   | 5   | 117 | 251 | 255 | 241 | 255 | 247 | 255 | 241 | 162 | 17  | 0   | 7   |
| 0  | 0   | 0   | 4   | 58  | 251 | 255 | 246 | 254 | 253 | 255 | 120 | 11  | 0   | 1   |
| 0  | 0   | 4   | 97  | 255 | 255 | 255 | 248 | 252 | 255 | 244 | 255 | 182 | 10  | 0   |
| 0  | 22  | 206 | 252 | 246 | 251 | 241 | 100 | 24  | 113 | 255 | 245 | 255 | 194 | 9   |
| 0  | 111 | 255 | 242 | 255 | 158 | 24  | 0   | 0   | 6   | 39  | 255 | 232 | 230 | 56  |
| 0  | 218 | 251 | 250 | 137 | 7   | 11  | 0   | 0   | 0   | 2   | 62  | 255 | 250 | 125 |
| 0  | 173 | 255 | 255 | 101 | 9   | 20  | 0   | 13  | 3   | 13  | 182 | 251 | 245 | 61  |
| 0  | 107 | 251 | 241 | 255 | 230 | 98  | 55  | 19  | 118 | 217 | 248 | 253 | 255 | 52  |
| 0  | 18  | 146 | 250 | 255 | 247 | 255 | 255 | 255 | 249 | 255 | 240 | 255 | 129 | 0   |
| 0  | 0   | 23  | 113 | 215 | 255 | 250 | 248 | 255 | 255 | 248 | 248 | 118 | 14  | 12  |
| 0  | 0   | 6   | 1   | 0   | 52  | 153 | 233 | 255 | 252 | 147 | 37  | 0   | 0   | 4   |
| 0  | 0   | 5   | 5   | 0   | 0   | 0   | 0   | 14  | 1   | 0   | 6   | 6   | 0   | 0   |

# Redes Neuronais

## Redes neuronais multilayer (feedforward)

- Camada de entrada =  $20 * 20 = 400$



### Matriz de entrada:

Cada coluna: 1 dígito ->  $20 \times 20 = 400$  entradas (linhas)

Nº de colunas = numero de exemplos =

11 exemplos \* 10 dígitos = 110 colunas

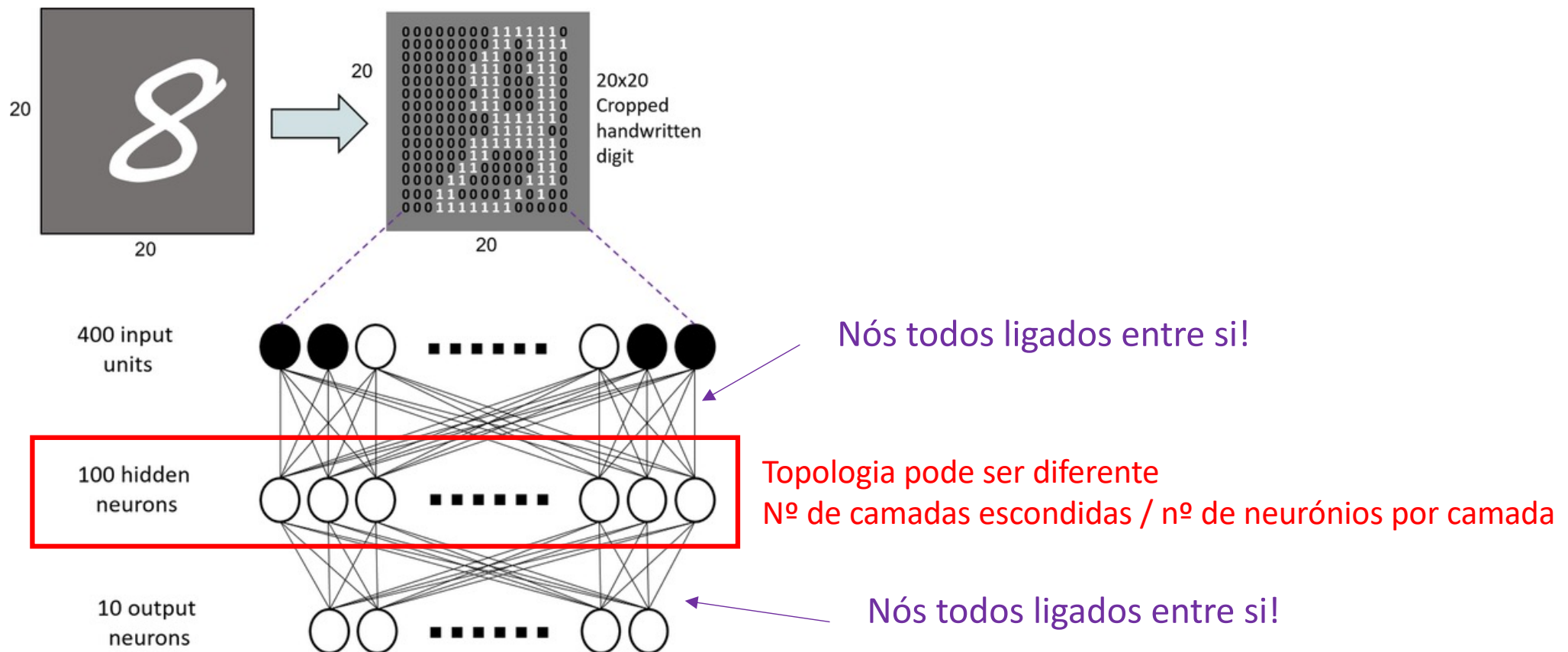
|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 0 |
| . | . | . | . | . | . |
| . | . | . | . | . | . |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 0 | 1 |
| . | . | . | . | . | . |
| . | . | . | . | . | . |
| . | . | . | . | . | . |
| 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 1 |

.....

# Redes Neuronais

## Redes neuronais multilayer (feedforward)

- Camadas escondidas: encontrar a melhor topologia!!

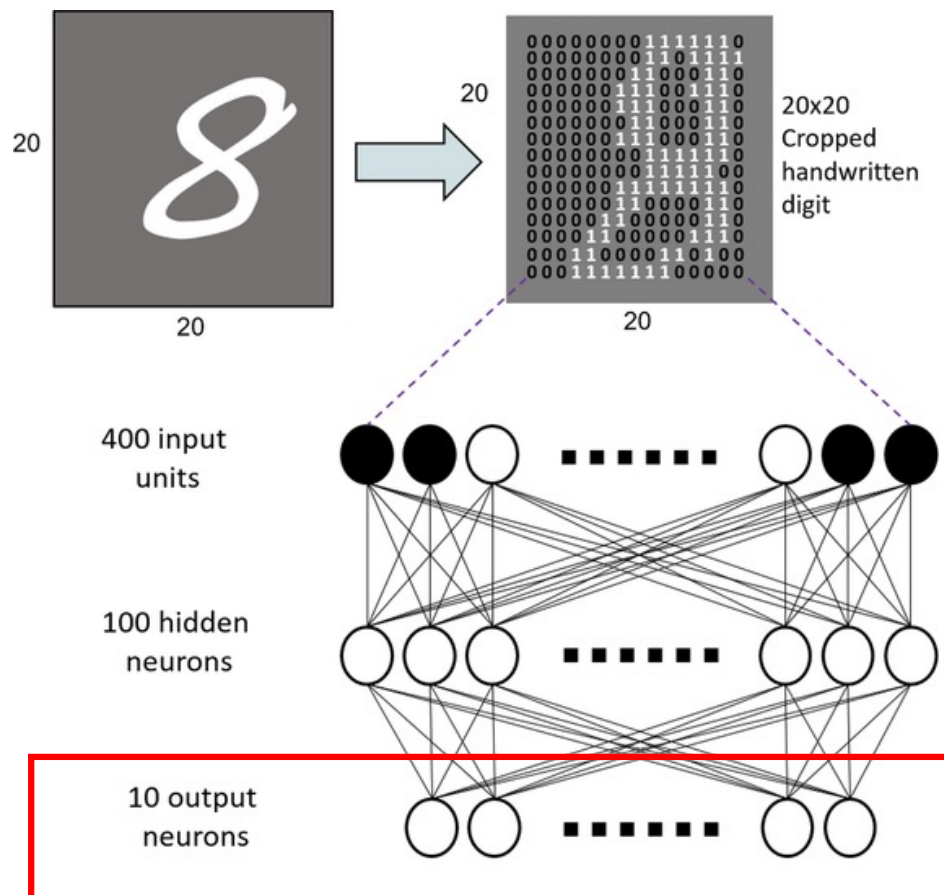




# Redes Neurais

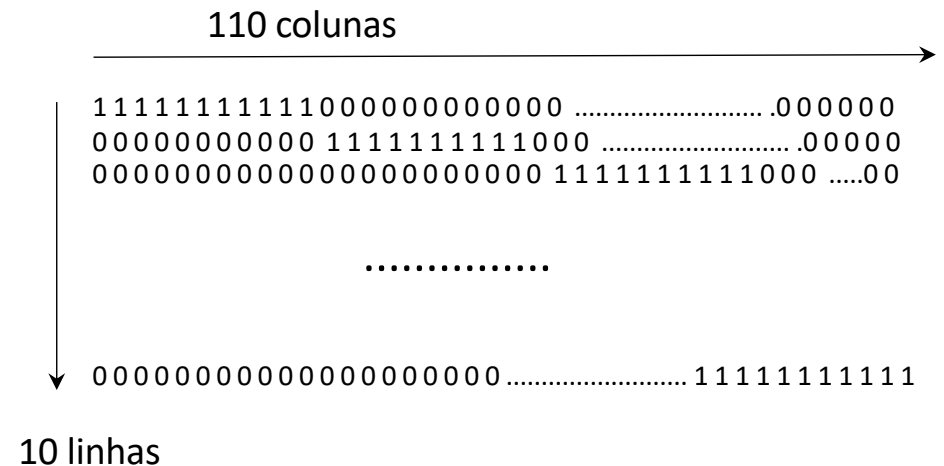
## Redes neuronais multilayer (feedforward)

- Camada de saída = 10 neurónios



### Matriz de saída:

Cada coluna reservada da para cada imagem: 110 colunas  
Nº de linhas = número de classes = 10 dígitos = 10 colunas





# Redes Neurais:

## Redes neuronais multilayer (feedforward)

- Pesos são inicializados aleatoriamente
- Para cada exemplo do dígito **0** aplicar o algoritmo de *backpropagation* até critério de convergência
  - Erro = 0
  - Convergência (iterações em alteração)
  - Iterações = valor estipulado
- Repetir para todos os exemplos dos dígitos **1, 2, ..., 9**
- No fim do treino os pesos foram ajustados de forma a reconhecer qualquer dígito dado no treino
- Teste da rede: fornecer novos dígitos:
  - Generalização?
  - Overfitting?

# Redes Neurais:

## Redes neurais multilayer (feedforward)

- Capacidade de generalização:

Quando após o treino a rede consegue generalizar o conhecimento, dando respostas corretas para exemplos até então desconhecidos.

- *Overfitting (sobreposição)*:

Manifesta-se quando um algoritmo se adapta excessivamente ou até mesmo de forma precisa aos dados de treino, levando a um modelo que não consegue fazer classificações corretas com novos dados.

O *overfitting* anula o propósito da aprendizagem. Uma rede com *overfitting* perde a capacidade de generalização

# Redes Neurais:

## Redes neurais multilayer (feedforward)

- Como evitar o overfitting:
  - Aumentar o número de exemplos
  - Dividir os exemplos corretamente: treino, validação e teste
  - Usar técnicas de *cross validation* na fase de treino
  - Usar técnicas de regularização
  - Experimentar diferentes topologias da rede

# Redes Neurais

## Redes neuronais multilayer (feedforward)

### Exemplo de generalização:

Avaliar o desempenho da rede na fase de treino: 85% de precisão

Avaliar o desempenho da rede com os novos exemplos (teste): 90% de precisão

| exemplos de treino    | exemplos de teste | Target (saída desejada) |
|-----------------------|-------------------|-------------------------|
| 0 0 0 0 0 0 0 0 0 0 0 | 0 0 0 0 0         | → 0                     |
| 1 1 1 1 1 1 1 1 1 1 1 | 1 1 1 1 1         | → 1                     |
| 2 2 2 2 2 2 2 2 2 2 2 | 2 2 2 2 2         | → 2                     |
| 3 3 3 3 3 3 3 3 3 3 3 | 3 3 3 3 3         | → 3                     |
| 4 4 4 4 4 4 4 4 4 4 4 | 4 4 4 4 4         | → 4                     |
| 5 5 5 5 5 5 5 5 5 5 5 | 5 5 5 5 5         | → 5                     |
| 6 6 6 6 6 6 6 6 6 6 6 | 6 6 6 6 6         | → 6                     |
| 7 7 7 7 7 7 7 7 7 7 7 | 7 7 7 7 7         | → 7                     |
| 8 8 8 8 8 8 8 8 8 8 8 | 8 8 8 8 8         | → 8                     |
| 9 9 9 9 9 9 9 9 9 9 9 | 9 9 9 9 9         | → 9                     |

# Redes Neurais

## Redes neuronais multilayer (feedforward)

### Exemplo de overfitting:

Avaliar o desempenho da rede na fase de treino: 100% de precisão

Avaliar o desempenho da rede com os novos exemplos (teste): 10% de precisão

| exemplos de treino    | exemplos de teste | Target (saída desejada) |
|-----------------------|-------------------|-------------------------|
| 0 0 0 0 0 0 0 0 0 0 0 | 0 0 0 0 0         | → 0                     |
| 1 1 1 1 1 1 1 1 1 1 1 | 1 1 1 1 1         | → 1                     |
| 2 2 2 2 2 2 2 2 2 2 2 | 2 2 2 2 2         | → 2                     |
| 3 3 3 3 3 3 3 3 3 3 3 | 3 3 3 3 3         | → 3                     |
| 4 4 4 4 4 4 4 4 4 4 4 | 4 4 4 4 4         | → 4                     |
| 5 5 5 5 5 5 5 5 5 5 5 | 5 5 5 5 5         | → 5                     |
| 6 6 6 6 6 6 6 6 6 6 6 | 6 6 6 6 6         | → 6                     |
| 7 7 7 7 7 7 7 7 7 7 7 | 7 7 7 7 7         | → 7                     |
| 8 8 8 8 8 8 8 8 8 8 8 | 8 8 8 8 8         | → 8                     |
| 9 9 9 9 9 9 9 9 9 9 9 | 9 9 9 9 9         | → 9                     |

# Recursos sobre Redes feedforward e backpropagation

- <https://www.youtube.com/watch?v=aircAruvnKk>
- <https://www.youtube.com/watch?v=IHZwWFHWa-w>
- <https://www.youtube.com/watch?v=llg3gGewQ5U>
- <https://www.youtube.com/watch?v=tleHLnjs5U8>